



INVENTIX · TEAM 4

API contract

Every endpoint the backend must provide, what it receives and returns, and the rule it enforces. Derived from the frontend, which was built first.

Source docs/08-api-contract.md

Companion 09-database-build.md

Generated 2 September 2026

Contents

How to read this document

Conventions

- Base URL
- Authentication
- Roles
- Ownership beyond role
- Error shape
- Status codes
- Idempotency
- Pagination

The three files behind every endpoint

Endpoints

shared/auth

- POST /auth/profile — any authenticated
- GET /auth/profile — any authenticated

shared/catalog

- GET /catalog/search?q=<text> — any authenticated

shared/notifications

- GET /notifications — any authenticated
- POST /notifications/{id}/read — any authenticated
- POST /notifications/device — any authenticated

customer/stocks

- GET /customer/stocks — customer
- GET /customer/stocks/summary — customer
- GET /customer/stocks/seasonal — customer
- GET /customer/stocks/{id} — customer
- GET /customer/stocks/{id}/adjustments — customer
- POST /customer/stocks — customer
- PATCH /customer/stocks/{id} — customer
- POST /customer/stocks/{id}/adjust — customer

customer/suppliers

- GET /customer/suppliers?q=<text> — customer
- GET /customer/suppliers/by-product/{catalog_product_id} — customer
- GET /customer/suppliers/{id} — customer

customer/ordering

- POST /customer/ordering/message — customer
- POST /customer/ordering/send — customer

customer/delivery

GET /customer/delivery — customer
GET /customer/delivery/{id} — customer
POST /customer/delivery/{id}/confirm-receipt — customer
POST /customer/delivery/{id}/cancel — customer
POST /customer/delivery/{id}/advance — customer

customer/uploads

POST /customer/uploads — customer
POST /customer/uploads/{id}/mapping — customer
GET /customer/uploads/{id}/unmatched — customer
POST /customer/uploads/{id}/unmatched — customer
POST /customer/uploads/{id}/apply — customer

customer/reports

GET /customer/reports — customer

supplier/listings

GET /supplier/listings — supplier
GET /supplier/listings/{id} — supplier
POST /supplier/listings — supplier
PATCH /supplier/listings/{id} — supplier

supplier/orders

GET /supplier/orders — supplier
GET /supplier/orders/{id} — supplier
POST /supplier/orders/{id}/confirm — supplier
POST /supplier/orders/{id}/reject — supplier

supplier/delivery

POST /supplier/delivery/{id}/advance — supplier
POST /supplier/delivery/{id}/delivered — supplier

Gaps

How to add an endpoint

Build order

Every endpoint the backend must provide, what it receives, what it returns, and which rule it enforces. This document is the backend's to-do list.

How to read this document

The contract was not designed in the abstract. It was **derived from the frontend**, which was built first. Every endpoint below already has a caller in `frontend/src/api/`, and every response shape already has a TypeScript interface in `frontend/src/types/api.ts` that a screen reads.

That means three things:

- **Nothing here is speculative.** If an endpoint is listed, a screen breaks without it.
- **The response shape is not negotiable.** Changing a field name here means changing `frontend/src/types/api.ts` and every screen that reads it, in the same sitting.
- **There is no API design phase left.** Only implementation.

Each entry names the file that owns it, so `docs/03-bug-map.md` and this document agree.

Conventions

Base URL

The app reads `EXPO_PUBLIC_API_URL` from its environment and prefixes every path with it. On a phone, `localhost` is the phone itself, so during development this must be the LAN address of the machine running FastAPI — for example `http://192.168.1.5:8000`. See `docs/06-open-questions.md` Q10.

While that variable is unset, `frontend/src/api/client.ts` serves fixture data instead of calling the backend, which is how the app runs today.

Authentication

Every request except none carries a Supabase access token:

```
Authorization: Bearer <supabase access token>
```

`frontend/src/api/client.ts` attaches this automatically from the current session. The backend must decode it in `backend/app/core/security.py` and resolve the caller's `user_id` and `role` **from the token and the profiles table**, never from anything in the request body.

Specification §15.2 is explicit about this: the app may send a request; it may not decide whether it is allowed. If an endpoint needs to know who is asking, it asks the token.

Roles

Every endpoint below is marked with who may call it:

Marking	Meaning
customer	Caller's <code>profiles.role</code> must be <code>customer</code> . Anything else is 403.
supplier	Caller's <code>profiles.role</code> must be <code>supplier</code> . Anything else is 403.
any	Any authenticated user.

This check belongs in `backend/app/dependencies.py` as a FastAPI dependency, applied in `routes.py` . It never belongs inside `service.py` , which does not know what an HTTP role is.

Hiding a tab in the app is a convenience for the user, not a security control (§4.2).

Ownership beyond role

Role is not enough. A customer may only read *their own* stock; a supplier may only confirm an order *addressed to them*. Ownership is checked twice, deliberately:

1. In `service.py` , which filters by the caller's id.
2. In the database, by row level security, which refuses the row even if the service forgets.

The second check is the one that actually protects the data. See `docs/09-database-build.md` .

Error shape

Every failure returns the same JSON body, produced by `backend/app/core/exceptions.py` :

```
{ "detail": "Lanka Traders has only 30 units of Nadu Rice 5kg.", "code": "quantity_unavailable" }
```

`detail` is written **for a shop owner, not a developer**. It is displayed directly by `frontend/src/components/ErrorBanner.tsx` . `code` is optional and machine-readable.

`frontend/src/lib/errors.ts` falls back to a generic message per status when `detail` is empty, so a blank error is never shown — but relying on that fallback is a bug, not a feature.

Status codes

Code	Used for
200	Success with a body
201	A resource was created
204	Success with no body — used by every action endpoint that returns nothing
400	Malformed request
401	Missing, expired or invalid token
403	Valid token, wrong role, or not your row
404	Not found, or found but not yours (see below)
409	A rule refused the change — illegal transition, duplicate upload
422	FastAPI validation failure against the Pydantic schema
500	Unhandled

404 versus 403 for someone else's row. Return 404. Returning 403 confirms the row exists, which tells a supplier that a given order id is real. Not found and not yours look identical from outside.

Idempotency

One endpoint requires it — `POST /customer/ordering/send` (§15.4). The app generates a key per popup session and replays it on retry:

```
Idempotency-Key: m8x2k9qp4a7f
```

`backend/app/core/idempotency.py` stores the key with the resulting `order_id`. A repeat of the same key returns the original order rather than creating a second one. Without this, a customer on a poor connection who taps Send twice gets two orders and double stock.

Pagination

Not used anywhere in this build, deliberately. A shop carries hundreds of products, not hundreds of thousands, and the supplier Orders feed filters client-side. If the supplier order volume grows past a few hundred, `GET /supplier/orders` is the first endpoint to page, and this row is where that decision gets recorded.

The three files behind every endpoint

Every endpoint is implemented across exactly three files, and the symptom tells you which one before you open anything (`docs/01-architecture.md`):

File	Does	Never does
<code>routes.py</code>	Declares the path, method, status code and role dependency. Parses the request, calls the service, returns the result.	Business logic. Database queries.
<code>service.py</code>	The work: queries, orchestration, calls into <code>domain/</code> .	Anything HTTP. It does not know what 404 means.
<code>schemas.py</code>	Pydantic models in and out.	Logic of any kind.

So an endpoint that 404s is a `routes.py` problem; an endpoint that returns the right shape with wrong numbers is a `service.py` problem; a field the app says is `undefined` is a `schemas.py` problem.

Endpoints

shared/auth

Owned by `backend/app/feeds/shared/auth/`. Specification §4.1, §4.2.

Sessions do **not** go through the backend. Sign-up, sign-in, token refresh and password reset all go from the app straight to Supabase Auth through `frontend/src/lib/supabase.ts`. Routing them through FastAPI would mean the backend handling raw passwords, which is worse.

The backend owns exactly one thing here: the `profiles` row, because it carries `role`.

POST /auth/profile — any authenticated

Creates the caller's profile after Supabase Auth has created the user. This is the only place a role is ever written.

Request:

```
{
  "role": "customer",
  "business_name": "Wasantha Kade",
  "contact_person": "Mahinda",
  "phone": "077 123 4567",
  "whatsapp_number": "077 123 4567",
  "address": "No 44, Horowpathana Road",
  "city": "Galle",
  "delivery_areas": ["Colombo", "Gampaha"]
}
```

`delivery_areas` is suppliers only (§4.1 step 6) and must be rejected on a customer.

Response `201`: the created profile as `AuthProfile` — `id`, `role`, `business_name`, `contact_person`, `email`, `phone`.

Rules:

- `id` comes from the token, never the body. A client that can choose its own profile id can overwrite someone else's.
- The row is written with the **service role key**, because RLS forbids the client inserting into `profiles` at all. That refusal is what makes this endpoint meaningful rather than decorative.
- 409 if a profile already exists for this user.

`GET /auth/profile` — any authenticated

Returns the caller's own profile. Used on launch to decide which navigation graph loads.

Response `200` : `AuthProfile` . `404` if the auth user exists but the profile row does not, which means registration was interrupted between the two steps — the app treats that as signed out.

Not built, and needed later: there is no endpoint to change a role. §4.2 says changing it requires support intervention, so it stays a manual database operation on purpose.

shared/catalog

Owned by `backend/app/feeds/shared/catalog/` . Specification §5.2.

`GET /catalog/search?q=<text>` — any authenticated

Searches the shared product catalog. Both customers and suppliers use it — customers to add a stock item, suppliers to add a listing. That shared use is the entire reason the catalog exists: if a customer typed "Rice 5kg" and a supplier typed "5kg rice bag", nothing could ever be matched.

Response `200` : array of `CatalogProduct` — `id` , `name` , `category` , `pack_size` , `unit` , `barcode` , `is_seasonal` , `is_active` .

Rules:

- Only `is_active` rows are returned.
- An empty `q` returns the full catalog, which is how the picker screens open.
- Match on `name` case-insensitively; matching `category` too is a cheap improvement.

Not built, and required by §5.2 and open question Q1: a "request a product" endpoint that creates a pending catalog entry when a user cannot find what they need. Both

`frontend/app/(customer)/stocks/add.tsx` and `frontend/app/(supplier)/listings/add.tsx` already show the button; it does nothing yet.

shared/notifications

Owned by `backend/app/feeds/shared/notifications/` . Specification §13.

`GET /notifications` — any authenticated

The caller's own notification history, newest first.

Response `200` : array of `AppNotification` — `id` , `user_id` , `type` , `title` , `body` , `related_order_id` , `related_stock_item_id` , `read_at` , `created_at` .

The two `related_*` fields are what let a tap open the right screen. `frontend/src/hooks/useNotifications.ts` routes on them, and push handling will use the same function.

`POST /notifications/{id}/read` — any authenticated

Sets `read_at` . Response `204` . `404` if the notification is not the caller's.

`POST /notifications/device` — any authenticated

Registers a Firebase token for push.

Request: `{ "fcm_token": "...", "platform": "ios" }`

`platform` is required and must be `ios` or `android` — Firebase needs to know which (§5.12). Upsert on the token so re-registering the same device does not create duplicate rows, and refresh `last_seen_at` so `jobs/` can expire stale tokens.

Response `204` .

customer/stocks

Owned by `backend/app/feeds/customer/stocks/` . Specification §6.1–§6.4, §6.7.

`GET /customer/stocks` — customer

Every stock item the caller owns, joined to its catalog product.

Response `200` : array of `StockItemView` :

```
{
  "id": "...",
  "product": { "id": "...", "name": "Highland Milk Powder", "category": "Dairy",
    "pack_size": "400 g", "unit": "packet", "barcode": null,
    "is_seasonal": false, "is_active": true },
  "quantity_on_hand": 12,
  "low_threshold": 25,
  "restock_requested": false,
  "preferred_supplier_id": "...",
  "preferred_supplier_name": "Lanka Traders (pvt) Ltd",
  "unit_price": 1180,
  "status": "low_stock"
}
```

`status` is computed by the backend, not the app. This matters more than it looks. The pie chart and the two list sections both read it, so if the app derived it independently they could disagree — the chart saying four items are low while the list shows three. One rule, one place:

```
quantity_on_hand > low_threshold          → "in_stock"
quantity_on_hand <= low_threshold AND no open order → "low_stock"
quantity_on_hand <= low_threshold AND open order   → "restock_requested"
```

That rule lives in `backend/app/domain/stock.py` and nowhere else.

`preferred_supplier_name` is denormalised into the response so the Low stock rows can say "Usually from Lanka Traders" without the app making a second request per row.

`GET /customer/stocks/summary` — customer

The three counts behind the pie chart.

Response `200`: `{ "total": 12, "in_stock": 6, "low_stock": 4, "restock_requested": 2 }`

Three segments, not four. The Figma draws an "Overstock" segment that cannot be computed — there is no `high_threshold` column to compute it from. See `docs/07-design-vs-spec.md`.

The counts must be derived from the same `classify` function as `status` above, or the chart and the list drift apart at exactly the moment someone is looking at both.

`GET /customer/stocks/seasonal` — customer

Upcoming festival warnings for this shop.

Response `200`: array of `SeasonalWarning` — `id`, `name`, `event_date`, `weeks_away`, `affected_categories`, and `products[]` of `{ stock_item_id, name, pack_size, suggested_quantity }`.

Rules, all in `backend/app/domain/seasonal.py` (§6.2):

- For each `seasonal_events` row, compare `event_date` to today.
- Include it when the gap is within `lead_time_months`.
- List only the caller's stock items whose category is in `affected_categories`.
- `suggested_quantity` is current quantity raised by `expected_uplift_pct`, rounded sensibly.

This is a date comparison. It is not machine learning, and it must not become machine learning — the learned version is a separate model in §7.2.

`GET /customer/stocks/{id}` — customer

One stock item, same `StockItemView` shape. `404` if not the caller's.

`GET /customer/stocks/{id}/adjustments` — customer

The audit trail for one item, newest first.

Response `200`: array of `StockAdjustment` — `id`, `stock_item_id`, `change_quantity`, `quantity_after`, `reason`, `source_id`, `created_by`, `created_at`.

This is the answer to "the number is wrong and I need to know why". §5.10 calls it not optional.

POST /customer/stocks — customer

Adds a stock item.

Request: `{ "catalog_product_id": "...", "quantity_on_hand": 120, "low_threshold": 30 }`

Response `201` . `409` if this customer already tracks that catalog product — one row per product per shop, enforced by a unique constraint, not by a check the service might forget.

PATCH /customer/stocks/{id} — customer

Edits the threshold or preferred supplier. Request is a partial: `{ "low_threshold": 40 }` or `{ "preferred_supplier_id": "..."` } .

Response `204` .

`quantity_on_hand` **must be rejected here**. A quantity may only change through the `adjust` endpoint below, so that every change writes an audit row. Accepting it here would create a path that changes stock without history, which is the one thing \$5.10 exists to prevent.

POST /customer/stocks/{id}/adjust — customer

Records a manual adjustment.

Request: `{ "change_quantity": -5, "reason": "damage" }`

`change_quantity` is signed. `reason` is one of `manual` , `damage` , `correction` from the app; `sales_upload` and `order_received` are written by the system, never by this endpoint.

Response `204` .

Rules:

- Goes through `domain/stock.py` , which calls the `apply_stock_adjustment` SQL function so the quantity update and the audit row commit together or not at all.
- Reject anything that would take `quantity_on_hand` below zero.
- If the change crosses the threshold, a low-stock notification fires (§13).

customer/suppliers

Owned by `backend/app/feeds/customer/suppliers/` . Specification §9, §12.

GET /customer/suppliers?q=<text> — customer

Company-name search. §9.1's second search type; results open a profile.

Response `200` : array of `SupplierView` :

```
{
  "id": "...", "business_name": "Lanka Traders (pvt) Ltd", "city": "Colombo 11",
  "is_active": true, "score": 92, "average_rating": 4.6, "rating_count": 38,
  "measured_delivery_days": 2.1, "is_new_supplier": false
}
```

`score` is out of 100, precomputed by the daily ranking job. It covers **only the two context-free components** — quality rating at 40 % and measured delivery speed at 30 % — renormalised to 100. Availability and price are per-order and cannot be scored without knowing what is being ordered, so they are returned as raw listing figures instead.

`measured_delivery_days` is the average of `purchased_at - requested_at` across that supplier's completed orders. It is **never** their stated `lead_time_days` (§12.1). Label it "measured" in any UI so it is visibly not a marketing number.

`is_new_supplier` is true below the configured minimum of completed orders, currently three. That threshold lives in `app_config`, not in code.

`GET /customer/suppliers/by-product/{catalog_product_id}` — customer

§9.1's product search: suppliers with an active listing for that product, **in ranking order**.

Response `200` : `SupplierView[]`, each carrying the matching listing:

```
"listing": { "unit_price": 1250, "quantity_available": 240,
             "lead_time_days": 2, "min_order_quantity": 10 }
```

The ordering is the backend's job. The app renders the array as given and must never re-sort it, or the ranking in §12 becomes decorative.

`GET /customer/suppliers/{id}` — customer

The full profile (§9.2). `SupplierProfileView` = `SupplierView` plus `contact_person`, `phone`, `whatsapp_number`, `email`, `address`, `delivery_areas`, `listings[]` (`ListingView`), and `order_history[]` (`OrderSummary`, this customer's own orders with them only).

Rules:

- Only the supplier's **public business fields** (§15.1). A customer must never see another customer's data through here, and `order_history` is scoped to the caller.
- Only active listings.

Gap — needs building. `frontend/src/api/suppliers.ts` has `searchForOrder()`, which is §9.3 selection mode: the same list, plus a `can_meet_quantity` flag marking suppliers who cannot fill the requested amount. It is currently computed **in the app** by comparing quantities client-side. That is wrong in the long run — the moment ranking accounts for availability, the app and the backend disagree. It should become `GET /customer/suppliers/for-order?product={id}&quantity={n}` and this row should move into the list above.

customer/ordering

Owned by `backend/app/feeds/customer/ordering/`. Specification §6.5. This is the most important feed in the application.

POST /customer/ordering/message — customer

Generates the restock message body.

Request:

```
{ "supplier_id": "...", "lines": [ { "stock_item_id": "...", "quantity": 60 } ] }
```

Response `200`: the message as a plain string.

Why this is server-side. The wording can then be improved without releasing a new version of the app, and one implementation cannot drift from another. `domain/message_builder.py` owns it; `backend/tests/test_message_builder.py` exists to be run after any wording change.

The message includes shop name, products and quantities, requested delivery date and notes.

POST /customer/ordering/send — customer

Creates the order. The single most consequential endpoint in the build.

Header: `Idempotency-Key: <string>` — **required**.

Request:

```
{
  "supplier_id": "...", "channel": "whatsapp",
  "message_body": "Hi Lanka Traders, ...", "message_edited": true,
  "requested_delivery_date": "2026-09-08", "notes": "Please call before delivery",
  "lines": [ { "stock_item_id": "...", "quantity": 60 } ]
}
```

Response `201`: `{ "order_id": "..." }`

What must happen, in one transaction (§6.5):

1. Re-validate every line against the supplier's listing. **The app's validation does not count.** Quantity above `quantity_available`, or below `min_order_quantity`, is a `409` naming the product and the limit — that message goes straight to the user, so it must be specific.
2. Create one `orders` row with status `requested`, and one `order_items` row per line, each capturing `unit_price_at_order` so a later price change does not rewrite history.
3. Set `restock_requested = true` on every stock item in the order. This is what stops the same product being ordered twice.
4. If `channel` is `in_app`, notify the supplier. Otherwise queue the WhatsApp or email send through `integrations/queue.py` — and if that service is down, **the order still exists** and the message retries

(§15.4). The user is told the order was placed and the message is pending; the whole action must not fail because a third party is unavailable.

`channel` may only be `in_app` when the supplier has an InventiX account. WhatsApp and email are how an unregistered supplier is ordered from at all (open question Q5).

customer/delivery

Owned by `backend/app/feeds/customer/delivery/`. Specification §8.

GET /customer/delivery — customer

Every order the caller placed, in any state. The app splits them into Requested and Confirmed (§8.1) and keeps rejected and cancelled on a separate history screen.

Response `200`: array of `OrderSummary` — `id`, `reference`, `status`, `channel`, `counterparty_name`, `counterparty_city`, `item_count`, `total_quantity`, `total_value`, `requested_at`, `requested_delivery_date`, `supplier_marked_delivered_at`, `rejection_reason`.

`counterparty_*` is the supplier here and the customer on the supplier side — one shape, two feeds, which is why `frontend/src/components/OrderCard.tsx` serves both.

`reference` is the human-readable `DEL-0001`. Generate it once and store it; do not derive it from a row number, or it changes when rows are deleted.

GET /customer/delivery/{id} — customer

Response `200`: `OrderDetailView` = `OrderSummary` plus `message_body`, `items[]`, `stage_history[]` of `{ status, at }`, `counterparty_phone`, `counterparty_address`, and `rating`.

`stage_history` is built from the timestamp columns on `orders` — `requested_at`, `confirmed_at`, `processing_at`, and so on. They exist as separate columns rather than a log table precisely so this is a single row read.

POST /customer/delivery/{id}/confirm-receipt — customer

The only endpoint that completes an order and the only one that increases stock.

Response `204`. What happens (§8.3, §11.4), all in one transaction:

1. Status `on_the_way` → `purchased`. Any other starting status is `409`.
2. Add each `order_items.quantity_requested` to its stock item.
3. Write a `stock_adjustments` row per item with reason `order_received`.
4. Set `restock_requested = false` on each, returning them to normal.
5. Record `purchased_at`, which completes the delivery-speed measurement for §12.
6. Notify the supplier, and prompt the customer to rate.

Step 5 is why a supplier cannot do this. If the supplier could mark their own delivery complete, the measurement of how fast they deliver would be self-reported (§10.4).

POST /customer/delivery/{id}/cancel — customer

Allowed from `requested` or `confirmed` only. Sets `cancelled`, clears `restock_requested` so the products return to Low stock, and returns reserved quantity to the supplier's listing if the order had been confirmed. `409` from any other status.

POST /customer/delivery/{id}/advance — customer

Request: `{ "status": "on_the_way" }`

Only for orders whose `channel` is `whatsapp` or `email`, where the supplier is updating the customer outside the app (§8.3). On an `in_app` order this is `403` — the supplier owns those transitions.

Validated by `domain/order_state_machine.py` like every other transition. A customer advancing manually still cannot skip a stage.

customer/uploads

Owned by `backend/app/feeds/customer/uploads/`. Specification §6.6. Four files rather than three, because parsing, matching and applying fail in three different ways.

Without this feed stock never decreases, nothing ever becomes low, and the application never triggers. It is not a nice-to-have.

POST /customer/uploads — customer

Uploads a CSV or Excel sales export.

This must be `multipart/form-data` carrying the actual file. The fixture in `frontend/src/api/uploads.ts` currently sends only `{ "file_name": "..."}` , which is enough to render the screens but is not the real contract — the backend has to hash the file contents, so it needs the contents.

`frontend/src/app/(customer)/stocks/upload/index.tsx` already picks a real file; only the request body needs changing when this endpoint lands.

Response `201`: `UploadSession` — `id`, `file_name`, `status`, `row_count`, `unmatched_count`, `columns[]`, and `suggested_mapping` of `{ product, quantity, date }`.

Rules:

- Hash the file and compare against this shop's previous uploads. A match is `409` with a message saying the report has already been applied. This is the only thing preventing stock being decremented twice from the same file (§15.4).
- `columns` is the header row, for the mapping screen.
- `suggested_mapping` is pre-filled from this shop's last upload, which is what makes mapping a one-time cost rather than a chore on every upload.
- Parsing lives in `parser.py`; dates and numbers in a Sri Lankan POS export will not be ISO.

POST /customer/uploads/{id}/mapping — customer

Request: `{ "product": "Item Name", "quantity": "Qty Sold", "date": "Sale Date" }`

Saves the column mapping and re-runs matching. Response `204` .

GET /customer/uploads/{id}/unmatched — customer

POS product names that resolved to nothing.

Response `200` : array of `{ pos_product_name, occurrences, catalog_product_id }` .

`occurrences` lets the owner deal with the rows that matter first.

POST /customer/uploads/{id}/unmatched — customer

Request: `{ "pos_product_name": "RICE-NADU-5KG", "catalog_product_id": "..." }`

Writes a `pos_product_aliases` row. That alias is permanent and reused on every future upload (\$5.9), which is why this list shrinks to nothing over a few months. A null `catalog_product_id` means skip this name.

Response `204` .

POST /customer/uploads/{id}/apply — customer

Commits the upload. Response `204` .

In one transaction (`applier.py`):

1. Write `sales_records` rows.
2. Reduce each matched stock item.
3. Write a `stock_adjustments` row per reduction with reason `sales_upload` and `source_id` set to the upload.
4. Set the upload to `applied` .
5. Fire low-stock notifications for anything that crossed its threshold.

All or nothing. A half-applied upload leaves quantities that cannot be explained or reversed.

customer/reports

Owned by `backend/app/feeds/customer/reports/` . Specification §7.

GET /customer/reports — customer

Returns the five report sections as metadata only — `key` , `title` , `description` , `icon` .

No figures and no charts. §7 builds this feed last because a report needs history to report on, and there is none until the other feeds have been in use. The screens render the sections with an empty state explaining what has to happen first.

The five (§7.1), each a plain query once data exists: stock movement over time, best and worst sellers, spend by supplier, order history with delivery times, and stock-out counts.

The four models in §7.2 come after that, computed on a schedule and stored — never on screen open.

supplier/listings

Owned by `backend/app/feeds/supplier/listings/` . Specification §10.1.

The tab says Stocks; the folder is `listings/` , because what a supplier manages is `supplier_listings` , not stock.

GET /supplier/listings — supplier

The caller's own listings, active and inactive.

Response `200` : array of `ListingView` — `id` , `product` , `quantity_available` , `unit_price` , `min_order_quantity` , `lead_time_days` , `is_active` , `is_low` .

`is_low` drives the supplier's own low-availability warning (§10.1) — they need to know when to restock themselves, because a customer cannot order what they do not hold.

GET /supplier/listings/{id} — supplier

POST /supplier/listings — supplier

Request: `{ catalog_product_id, quantity_available, unit_price, min_order_quantity, lead_time_days }`

`409` if this supplier already lists that product — one row per product per supplier.

PATCH /supplier/listings/{id} — supplier

Partial update of any of the above, plus `is_active` .

Setting `is_active: false` is how a listing is retired. There is no delete. Order history references listings, and deleting one would break every past order that used it (§10.1).

Response `204` on all three writes. `404` on someone else's listing, never 403.

supplier/orders

Owned by `backend/app/feeds/supplier/orders/` . Specification §10.2.

GET /supplier/orders — supplier

Every order ever received, in any state. The app splits them into Pending, Active and History and filters client-side.

Response `200` : `OrderSummary[]` , with `counterparty_*` being the customer.

An order appears here permanently. It additionally appears in the Delivery queue only while active — which is why these are two feeds divided by purpose rather than one filtered by stage. Splitting by stage would make an order vanish from one feed and reappear in another the instant it was confirmed (§10).

GET /supplier/orders/{id} — supplier

`OrderDetailView`, including the customer's delivery address and contact details — the supplier needs them to actually deliver — and any rating given.

POST /supplier/orders/{id}/confirm — supplier

Accepts the order. Response `204`. In one transaction:

1. Status `requested` → `confirmed`. Anything else is `409`.
2. **Re-check availability against every listing in the order.** If availability has fallen since the order was placed, refuse with `409` and a message saying so — only Reject remains available. §10.2 is explicit: a supplier must never be able to confirm stock they no longer hold.
3. Reduce `quantity_available` on each listing, so they are not shown as holding stock they have committed.
4. Notify the customer.

POST /supplier/orders/{id}/reject — supplier

Request: `{ "reason": "Out of stock until the middle of next month." }`

`reason` is **required** and non-trivial. It is stored on the order and shown to the customer, who has to decide what to do next; a vague reason wastes their time.

Sets `rejected`, clears `restock_requested` so the products return to Low stock, and notifies the customer with an offer to re-order elsewhere (§11.4). Response `204`.

supplier/delivery

Owned by `backend/app/feeds/supplier/delivery/`. Specification §10.3, §11.3.

Reads the same `orders` table as the feed above with a different status filter — §10.5 confirms this adds no schema. The frontend accordingly serves both from `frontend/src/api/orders.ts`.

POST /supplier/delivery/{id}/advance — supplier

Request: `{ "status": "processing" }`

Moves the order one stage. Legal steps only (§11.2):

```
confirmed → processing → put_to_delivery → on_the_way
```

Validated by `domain/order_state_machine.py`, which is also what the customer's manual-advance endpoint calls. One state machine, four callers — that is the entire reason `domain/` exists. `409` on an illegal transition. Response `204`.

POST /supplier/delivery/{id}/delivered — supplier

Records `supplier_marked_delivered_at` and notifies the customer to confirm receipt.

It does not change the status. The order stays `on_the_way` until the customer confirms.

This is deliberate and should be understood before anyone changes it (§11.3). Adding a seventh visible stage would make the pipeline longer than a shop owner wants to read, and letting the supplier set **purchased** directly would mean stock rising on a claim rather than a fact.

Response **204** .

Gaps

Things a screen already expects that no endpoint covers yet. Each is a real piece of work, not an oversight to argue about.

Gap	Needed by	Note
Upload endpoint takes a filename, not a file	\$6.6	Must become <code>multipart/form-data</code> . The backend hashes contents.
<code>can_meet_quantity</code> computed in the app	\$9.3	Move server-side as <code>GET /customer/suppliers/for-order</code> .
No "request a product" endpoint	\$5.2, Q1	Both add-product screens show the button already.
No rating submission endpoint	\$12.3	<code>RatingPrompt</code> collects a score and comment and currently discards them. This blocks 40 % of the ranking score.
No password change	menu	The menu item exists. Supabase Auth handles it client-side; no backend work.
Mock switch is global	—	<code>useMockData = !API_BASE_URL</code> in <code>client.ts</code> flips all twelve feeds at once. Make it per-feed before Phase D, or the app breaks the moment the first endpoint is real.

The rating gap is the one to fix early. Ratings accumulate slowly, and \$12.1 makes them 40 % of the ranking — a month of orders with no rating endpoint is a month of unrankable suppliers.

How to add an endpoint

1. Add the function to `frontend/src/api/<feed>.ts` and its types to `src/types/api.ts` .
2. Add the row to this document.
3. Implement `schemas.py` , then `service.py` , then `routes.py` — in that order, because each depends on the one before.
4. If the rule is used by more than one feed or by a job, it goes in `domain/` instead.
5. Update `docs/03-bug-map.md` if the file now owns a new symptom.

Build order

Phase	Feeds	Why here
1	core/ , dependencies.py , config.py	Everything imports it.
2	shared/auth	Nothing works signed out.
3	shared/catalog	Stocks and listings both pick from it.
4	customer/stocks	The home screen, and the trigger for everything else.
5	supplier/listings	Ordering needs something to order.
6	customer/suppliers	Needs listings to rank.
7	customer/ordering	Needs stocks, suppliers and listings.
8	customer/delivery , supplier/orders , supplier/delivery	One state machine, built together.
9	shared/notifications	Everything above has something to announce.
10	customer/uploads	Independent; can move earlier if someone is free.
11	customer/reports	Last. Needs history to report on.

domain/ is written alongside phases 4–8 and, being pure logic with no HTTP and no database, can be built and tested by a second person before the schema exists.